Test Plan: Contact Manager System

Software Testing, Validation and Verification (CSE265)

Project: C++ Console-based Contact Manager System

Under the Supervision of: Dr. Gehad Mohey

Version: 1.1

Date: May 4, 2026

Student Name:	ID:
Mazen Yasser	247861
Jasmine Ali	240247
Sandy Atef	245077
Mariam Mohamed Reyad	245171
Marwan Ehab	248889

Table of Contents

1. Introduction

This Test Plan describes the testing scope, strategy, objectives, and approach for the Contact Manager System. The goal is to ensure the system meets all functional and non-functional requirements specified in the SRS and adheres to the architectural design depicted in the UML Class and Sequence diagrams. In addition, evaluating the product quality and identifying key defects.

2. Objectives

- To verify all functional requirements specified in the SRS, including adding, viewing, searching, updating, and deleting contacts.
- To ensure data consistency on JSON file.
- To verify all non-functional requirements stated in the SRS.
- To identify bugs and inconsistencies

3. Scope

The testing Scope of this project includes:

- System testing, unit testing, integration testing, static testing, and dynamic testing.
- Static review and dynamic execution of all project-owned modules are included in scope, but external libraries are excluded from testing.
- Performance testing is excluded.

4. Risks Analysis

- Missing JSON file or data corruption in the JSON file due to unexpected termination during add, update, or delete operations .
  Likelihood: Medium
  Impact: Critical
- Failure to validate empty or invalid fields (e.g., missing name or invalid numbers).
  Likelihood: High
  Impact: Critical
- Duplicate contacts due to lack of uniqueness constraints.
  Likelihood: High
  Impact: High
- Duplicate assigned unique IDs.
  Likelihood: Low
  Impact: Critical
- Failure in search leading to missing or wrong results.
  Likelihood: Medium
  Impact: High
- Data inconsistency between shown and stored data.
  Likelihood: Medium
  Impact: Critical

5. Test Items

The following artifacts will be subject to testing:

- **Source Code:** All C++ modules including ContactConsole, ContactRepository, and ContactService.
- **Database File:** The contacts.json file used for persistence.
- **Documentation:** The Software Requirements Specification (SRS) and UML Design diagrams (Class and Sequence) for static review.

6. Features to be Tested

6.1 Functional Features

- **System Initialization:** Loading data from JSON file or initializing empty list.
- **Main Menu Navigation:** Correct display and routing of the menu options.
- **Add Contact:** Unique ID assignment and handling input.
- **View Contact:** Correct display of stored contacts and data.
- **Search Contact:** Full or partial name matching with stored contacts.
- **Update Contact:** Select by ID, modify, and store new data.
- **Delete Contact:** Select by ID and updating data.

6.2 Non-Functional Features

- **Usability:** Clarity of console prompts and structured output.
- **Input Validation:** Handling invalid inputs (e.g., empty fields or invalid ID).
- **Reliability & Persistence:** Correct read/write operations to JSON file .
- **Maintainability:** Proper layer separation (console, logic ,repository).
- **Testability:** Independent testable layers.

7. Approach (Testing Strategy)

7.1 Test Types

- **Static Testing:** Conduct **Manual Technical Review** of the SRS and C++ source code to identify logic flaws or deviations from the UML design before execution and to verify that design constraints are met.
- **Dynamic Testing:** Execute the compiled program with specific test data to validate behavior against requirements FR-IN through FR-DC .

7.2 Levels of Testing

- **Unit Testing:**
  - What: Individual classes
  - Technique: **White-Box Testing** (Statement and Branch coverage).
- **Integration Testing:**
  - What: Interaction between Modules and the nlohmann/json library to ensure data flows correctly from the JSON file to the objects and vice versa.
  - Technique: **Black-Box Testing** focusing on interface integrity.
- **System Testing:**
  - What: The entire Contact Manager Console application from initialization to exit. Console Prompts and Messages.
  - Technique: **Black-Box Testing** (Functional testing based on SRS requirements).

8. Item Pass/Fail Criteria

- **Pass:** The system performs the function as described in the SRS (FR-IN through FR-DC) and the JSON database reflects the change correctly.
- **Fail:** Any deviation from the SRS, such as:
    - Displaying wrong or invalid data.
    - Failing to update the JSON file after any operation.
    - Accepting invalid inputs.

9. Testing Enter/Exit Criteria:

9.1. Entry Criteria

Before testing can begin, the following conditions must be met to avoid wasting resources on an unstable system:

- **Completion of Development:** All C++ modules (ContactConsole, ContactRepository, ContactService, etc.) must be coded and compiled without errors.
- **Availability of Test Artifacts:** The Test Plan and Test Case Specifications must be documented and reviewed.
- **Environment Readiness:** A C++11 (or later) compiler and the nlohmann/json library must be correctly configured in the development environment.
- **Successful Static Review:** The source code must have passed a basic walkthrough to ensure it follows the logic depicted in the Sequence Diagram.

9.2. Exit Criteria

Testing for the Contact Manager Console System is considered complete when the following benchmarks are reached:

- **Requirement Coverage:** 100% of the functional requirements (FR-IN through FR-DC) have been executed and tested.
- **Bug Resolution:** All "Critical" and "High" severity defects are identified and noted.
- **Persistence Validation:** Every operation has been verified to correctly write back to the contact.json file.
- **Constraint Adherence:** The final codebase is confirmed to be under the 500-line limit and distributed across the planned modules.
- **Documentation Finalization:** The Test Log and Defect Reports are completed and signed off by the lead developer or tester.

10. Test Deliverables

- Test Plan (this document).
- Test Case Specifications.
- Reviews
- Test Log / Execution Report.
- Defect Reports.

11. Environmental Needs

- **Hardware:** Standard PC capable of running C++ console applications.
- **Software:** C++ compiler (C++11 or later).
- **Libraries:** nlohmann/json library integrated into the source folder.

12.References

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.

Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.

GoogleTest Team. (2025). *GoogleTest user's guide*. GitHub. https://google.github.io/googletest/

nlohmann, N. (2025). *JSON for Modern C++ documentation*. GitHub. https://json.nlohmann.me/

Microsoft. (2025). *Visual Studio documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/visualstudio/

IEEE. (2013). *IEEE standard for software and system test documentation (IEEE Std 829-2008)*. IEEE Standards Association. https://standards.ieee.org/

ISO/IEC/IEEE. (2017). *Systems and software engineering—Vocabulary (ISO/IEC/IEEE 24765:2017)*. ISO. https://www.iso.org/

The Open Source Initiative. (2025). *Introducing JSON*. https://www.json.org/json-en.html